

Project Title Proposal

PixelRack

A collection app that lets a Hot Wheels collector photograph each die-cast car they own, turn that photo into a pixel-art sprite in one consistent style, and arrange those sprites on themed virtual scenes, a wooden rack, a garage, a Japanese convenience store so the collection can be displayed and shown off without unboxing anything.

Who is it for

Who, specifically: Me, and collectors like me, someone with thirty-plus Hot Wheels cars, most of them still carded or stored in a bin because there is no shelf space to display them all at once.

What they are trying to get done in the moment they open it: Either (a) they just bought a new car and want to add it to the collection while the photo is still on their phone, or (b) they want to look at what they own, laid out as a display rather than as a spreadsheet — and rearrange which cars are “on show” in each scene.

Sections or routes this app needs

Multi-screen app using React Router.

#	Section / route	What it is for
1	Landing / hero (/)	The front door: the PixelRack emblem, what the app does, and a way into log in / sign up.
2	Auth (/login, /register)	Sign in or create an account, so one person's collection stays theirs. Two routes, one job.
3	Dashboard / Rack (/dashboard)	The main screen: upload a new car and see the whole collection as a grid of sprites on a wooden rack, sortable and filterable by series.
4	Virtual Garage (/garage)	A 2-slot scene, pick two cars from the collection to park in the garage, then click one to wash it.
5	7-Eleven Japan (/konbini)	A 3-slot scene, park three cars outside the konbini, click one to make it sparkle.

Rows 4 and 5 are the same screen shape with different artwork, slot counts and click effect; both render from one shared **ScenePage** component, so they cost far less than two screens' worth of work.

Test each one: Removing the Dashboard would kill the app, it is the only place a car gets added or seen in full. The scenes are the *point* of the app rather than the mechanics of it (the display, not the database), so they stay. Auth is the one I would park first if this needed to shrink: a single implicit user would still demonstrate everything except “your collection is yours.”

State: what data does the app hold?

Most important screen: **Dashboard (/dashboard)**.

Data	Shape (rough)	Who owns it	Changes when...
cars	[{ id, name, series, pixellImageUrl, originalImageUrl, createdAt }]	DashboardPage	loaded on mount; a car is uploaded; a car is deleted (optimistic, rolled back if the API fails)
environments	[{ id, name, slots, isPremium, sortOrder }]	DashboardPage	loaded once on mount
sort	'shelf' 'name'	DashboardPage	user changes the sort control in RackHeader
seriesFilter	'all' <series name>	DashboardPage	user picks a series in RackHeader
isLoading, error	boolean, string	DashboardPage	fetch starts / finishes / fails
visibleCars, seriesOptions, shelfCount	derived	DashboardPage (useMemo)	recomputed when cars , sort or seriesFilter change — not stored separately
file, previewUrl, crop, name, series	File, string, {x,y,w,h}, string, string	UploadPanel	user picks a photo, drags the crop box, types details
progress, isProcessing	number (0–100), boolean	UploadPanel	upload XHR reports progress
pixelationEnabled	boolean	UploadPanel	fetches from /api/config ; assumed false until the server says

			otherwise
placements	[{ slotIndex, car }]	ScenePage	scene loads; a car is placed in or cleared from a slot
pickingSlot, activeEffect	number null	ScenePage	user clicks an empty slot (opens picker) / clicks a placed car (plays effect)

State ownership note: `cars` lives in `DashboardPage` rather than in `Rack`, because both `Rack` (which displays it) and `UploadPanel` (which adds to it) need it — so it sits in their lowest common parent and is passed down as props, with `onUpload` and `onDelete` passed back up. No state-management library; the app is too small to need one.

What each screen contains

Screen: Dashboard / Rack

- **Block 1** — ``Navbar``: logo, links to the scenes, sign-out.
- **Block 2** — ``UploadPanel`` (left sidebar): drag-and-drop file input → `ImageCropper` (crop to the car itself, because background removal treats a packaged car as one object) → car name + series inputs → upload button → progress bar and a small transform queue.
- **Block 3** — ``RackHeader``: rack name (“The Wooden Shelf”), car count, shelf count, sort control, series filter.
- **Block 4** — ``Rack``: the grid of `CarSprite` tiles on the background shelf, each with a delete action.
- **Block 5** — ``EnvironmentGallery``: `EnvironmentThumb` cards, each showing real scene artwork and linking through to that scene.
- **Block 6** — ``Footer``.

Content you need to gather

- **Scene artwork** — wooden rack, virtual garage, 7-Eleven Japan storefront. Each needs its parking bays measured as percentage positions, so sprites stay pinned to the right spot as the scene scales. *(Done — all three exist.)*
- **Branding** — PixelRack emblem/logo, favicons, landing-page art, the pixel UI icons (house, rack, upload). *(Done.)*

- **Real car photos** — enough of my own collection to demo with, shot against a plain background where possible.
- **Fonts and palette** — the pixel display font and the 16-colour-ish UI palette, so the chrome matches the sprites.
- **Infrastructure** — a Neon Postgres database, Neon Auth for accounts, and (for the transformation step) a Gemini or Cloudflare Workers AI key with billing enabled.
- **Audio** — the background track used on the landing page.

One risk

The photo-to-sprite pixelation pipeline's quality and its cost.

Getting a photo to come out as *drawn-looking* sprite art is the part I am least sure of. Plain downscaling plus colour reduction does not produce pixel art from a photograph; it produces a small blurry photograph. The current build therefore sends the cropped photo to an image model (Gemini [gemini-2.5-flash-image](#), with a Cloudflare Stable Diffusion img2img path as an alternative and local background removal as a free fallback).

That creates the real risk, which is money and availability rather than code: image generation has **no free tier** and is billed roughly **\$0.04 per upload**, so the endpoint is deliberately gated behind [PIXELATION_ENABLED](#) and ships **switched off** in the demo deployment, an open endpoint on the public internet would spend real money on anyone who found the URL. The consequence is that the headline feature is the one thing a visitor cannot currently try.

What I need to resolve early: whether the free Cloudflare path is good enough to leave enabled in public, or whether the demo should ship with a small set of pre-generated sprites, so the scenes are still populated and the app is still demonstrable without the billed step.